



LECTURE 2

Pandas Bootcamp: Part 1

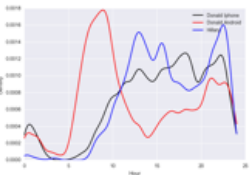
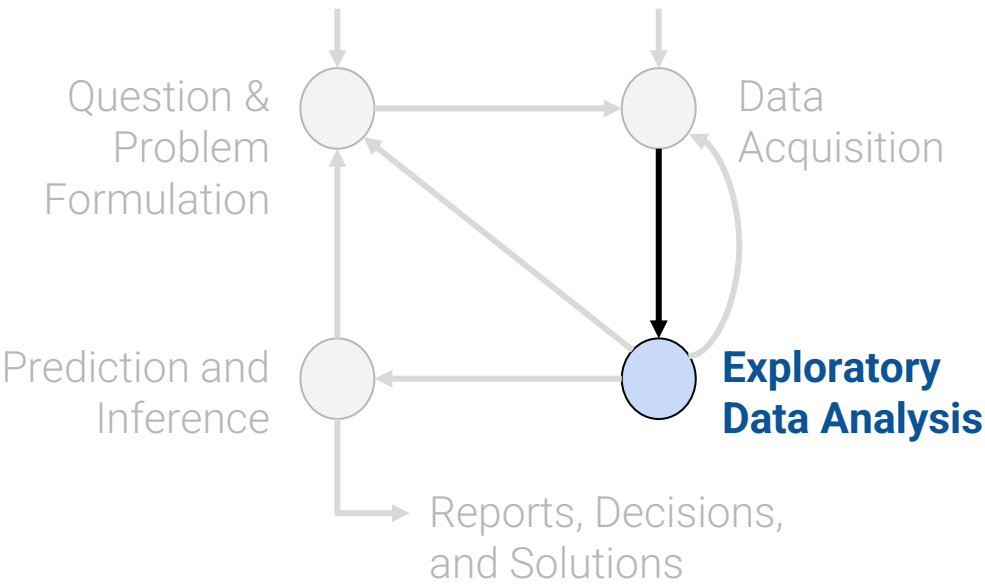
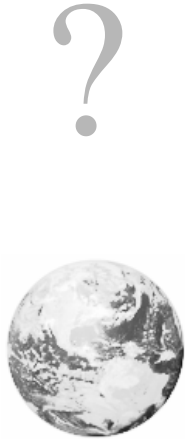
Intro to useful Pandas functions for Exploratory Data Analysis

CSCI 3022 @ CU Boulder

Dr. Osita Onyejekwe

Content credit: [Acknowledgments](#)

Plan for first 2 weeks



(Weeks 1)

EDA, Wrangling, and Data Visualization

Lesson Learning Objectives:

- Explain the relationship between DataFrames, Series and Indices in Pandas
- Understand and implement methods for extracting data: .loc, .iloc, and [].
- Understand and implement methods for conditional selection in Pandas
- Modify columns in a Pandas DataFrame
- Manipulate and transform Series and DataFrames using common utility functions:
 - (value_counts, describe, info, unique, shape, sort_values)

Roadmap

Lecture 2, CSCI 3022

- Pandas Bootcamp:
 - [Pandas Data Structures](#)
 - [Extracting Data](#)
 - [Conditional Selection](#)
 - [Modifying Columns](#)
 - [Useful Utility Functions](#)

[Supporting Material](#)

More with Conditional Selection

The DataFrame API

The API for the **DataFrame** class is enormous.

- API: "Application Programming Interface".
- The API is the set of abstractions supported by the class.

Full documentation is at

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

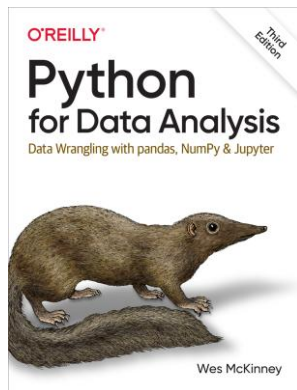
- We will only consider a tiny portion of this API.

We want you to get familiar with the real world programming practice of...
Googling!

- Answers to your questions are often found in the **pandas** documentation, Stack Overflow, etc.

Very Useful Resources:

- [Data Wrangling with Pandas Cheat Sheet](#)
- [Open Source Textbook: Python For Data Analysis](#)



Learning Objectives

- Understand the relationship between DataFrames and Indices in Pandas
- Create DataFrames
- Manipulate indices

Pandas Data Structures

- Pandas Bootcamp:
 - **Pandas Data Structures**
 - Extracting Data
 - Conditional Selection
 - Adding/Modifying/Deleting Columns
 - Useful Utility Functions

Supporting Material:

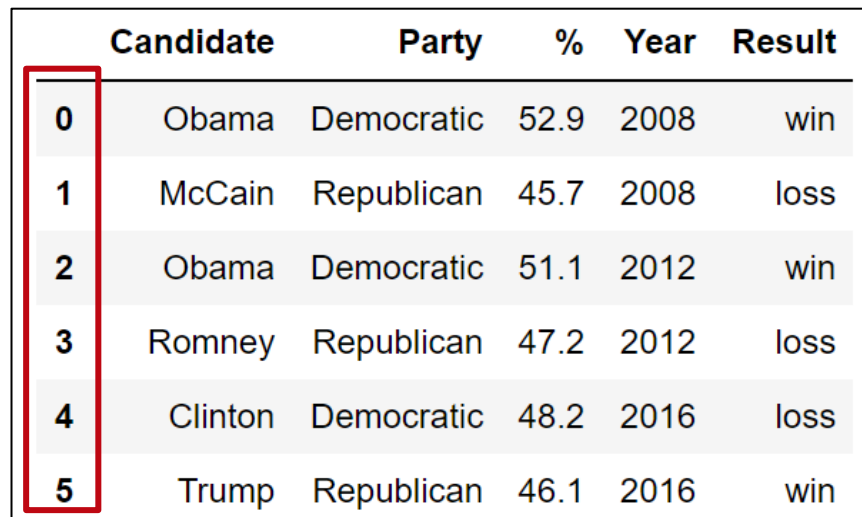
More with Conditional Selection

The Relationship Between DataFrames, Series, and Indices

We can think of a **DataFrame** as a collection of **Series** that all share the same **Index**.

- Candidate, Party, %, Year, and Result **Series** all share an **Index** from 0 to 5.

Candidate Series Party Series % Series Year Series Result Series



	Candidate	Party	%	Year	Result
0	Obama	Democratic	52.9	2008	win
1	McCain	Republican	45.7	2008	loss
2	Obama	Democratic	51.1	2012	win
3	Romney	Republican	47.2	2012	loss
4	Clinton	Democratic	48.2	2016	loss
5	Trump	Republican	46.1	2016	win

Indices Are Not Necessarily Unique

The row labels that constitute an index do not have to be unique.

- Left: The **index** values are all unique and numeric, acting as a row number.
- Right: The **index** values are named and non-unique.

	Candidate	Party	%	Year	Result
0	Obama	Democratic	52.9	2008	win
1	McCain	Republican	45.7	2008	loss
2	Obama	Democratic	51.1	2012	win
3	Romney	Republican	47.2	2012	loss
4	Clinton	Democratic	48.2	2016	loss
5	Trump	Republican	46.1	2016	win

Year	Candidate	Party	%	Result
2008	Obama	Democratic	52.9	win
2008	McCain	Republican	45.7	loss
2012	Obama	Democratic	51.1	win
2012	Romney	Republican	47.2	loss
2016	Clinton	Democratic	48.2	loss
2016	Trump	Republican	46.1	win

- We describe "labels" as the bolded text at the top and left of a `DataFrame`.

Row labels
(aka index)

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
177	2016	Jill Stein	Green	1457226	loss	1.073699
178	2020	Joseph Biden	Democratic	81268924	win	51.311515
179	2020	Donald Trump	Republican	74216154	loss	46.858542
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
181	2020	Howard Hawkins	Green	405035	loss	0.255731

Column labels

Accessing a DataFrame's labels: .index and .columns methods

```
elections = pd.read_csv("data/elections.csv", index_col="Year")
```

	Candidate	Party	Popular vote	Result	%
Year					
1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
1828	Andrew Jackson	Democratic	642806	win	56.203927
1828	John Quincy Adams	National Republican	500897	loss	43.796073
1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...
2016	Jill Stein	Green	1457226	loss	1.073699
2020	Joseph Biden	Democratic	81268924	win	51.311515
2020	Donald Trump	Republican	74216154	loss	46.858542
2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
2020	Howard Hawkins	Green	405035	loss	0.255731

```
elections.index
```

```
Index([2024, 2024, 2024, 2024, 2024, 2024, 2024, 2020, 2020, 2020,
      ...,
      1836, 1836, 1836, 1832, 1832, 1832, 1828, 1828, 1824, 1824],
      dtype='int64', name='Year', length=189)
```

```
elections.columns
```

```
Index(['Candidate', 'Party', 'Popular vote', 'Result', '%'], dtype='object')
```

The DataFrame elections with "Year" as Index

Creating a DataFrame

Many approaches exist for creating a **DataFrame**.

- **From a CSV file.** (most common method in our class)
- Using a list and column name(s).
- From a dictionary.
- From a **Series**.

```
elections = pd.read_csv("data/elections.csv")
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
177	2016	Jill Stein	Green	1457226	loss	1.073699
178	2020	Joseph Biden	Democratic	81268924	win	51.311515
179	2020	Donald Trump	Republican	74216154	loss	46.858542
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
181	2020	Howard Hawkins	Green	405035	loss	0.255731

See Supporting Materials from Lesson 1 for examples of creating DataFrames using the other 3 methods

The DataFrame **elections**

Indices Are Not Necessarily Row Numbers

```
# Creating a DataFrame from a CSV file and specifying the Index column
elections = pd.read_csv("data/elections.csv", index_col = "Candidate")
```

Candidate	Year	Party	Popular vote	Result	%
Andrew Jackson	1824	Democratic-Republican	151271	loss	57.210122
John Quincy Adams	1824	Democratic-Republican	113142	win	42.789878
Andrew Jackson	1828	Democratic	642806	win	56.203927
John Quincy Adams	1828	National Republican	500897	loss	43.796073
Andrew Jackson	1832	Democratic	702735	win	54.574789

Modifying Indices

- We can select a new column and set it as the index of the **DataFrame**.

Example: Setting the index to the "Party" column.

```
elections = elections.set_index("Party")
```

	Candidate	Year	Popular vote	Result	%
Party					
Democratic-Republican	Andrew Jackson	1824	151271	loss	57.210122
Democratic-Republican	John Quincy Adams	1824	113142	win	42.789878
Democratic	Andrew Jackson	1828	642806	win	56.203927
National Republican	John Quincy Adams	1828	500897	loss	43.796073
Democratic	Andrew Jackson	1832	702735	win	54.574789
...	...	...	...	...	...
Green	Jill Stein	2016	1457226	loss	1.073699
Democratic	Joseph Biden	2020	81268924	win	51.311515
Republican	Donald Trump	2020	74216154	loss	46.858542
Libertarian	Jo Jorgensen	2020	1865724	loss	1.177979
Green	Howard Hawkins	2020	405035	loss	0.255731

An Important Note: DataFrame Copies

Notice that we *re-assigned* `elections` to an updated value on the previous slide.

```
elections = elections.set_index("Party")
```

By default, **pandas** methods create a **copy** of the **DataFrame**, without changing the original **DataFrame** at all. To apply our changes, we must update our **DataFrame** to this new, modified copy.

```
elections.set_index("Party")
```

`elections`

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
177	2016	Jill Stein	Green	1457226	loss	1.073699
178	2020	Joseph Biden	Democratic	81268924	win	51.311515
179	2020	Donald Trump	Republican	74216154	loss	46.858542
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
181	2020	Howard Hawkins	Green	405035	loss	0.255731

Our change was not applied!



Resetting the Index

- We can change our mind and reset the **Index** back to the default list of integers.

```
elections = elections.reset_index()
```

	Candidate	Year	Popular vote	Result	%
Party					
Democratic-Republican	Andrew Jackson	1824	151271	loss	57.210122
Democratic-Republican	John Quincy Adams	1824	113142	win	42.789878
Democratic	Andrew Jackson	1828	642806	win	56.203927
National Republican	John Quincy Adams	1828	500897	loss	43.796073
Democratic	Andrew Jackson	1832	702735	win	54.574789
...	...	...	...	...	...
Green	Jill Stein	2016	1457226	loss	1.073699
Democratic	Joseph Biden	2020	81268924	win	51.311515
Republican	Donald Trump	2020	74216154	loss	46.858542
Libertarian	Jo Jorgensen	2020	1865724	loss	1.177979
Green	Howard Hawkins	2020	405035	loss	0.255731

	Candidate	Year	Party	Popular vote	Result	%
0	Andrew Jackson	1824	Democratic-Republican	151271	loss	57.210122
1	John Quincy Adams	1824	Democratic-Republican	113142	win	42.789878
2	Andrew Jackson	1828	Democratic	642806	win	56.203927
3	John Quincy Adams	1828	National Republican	500897	loss	43.796073
4	Andrew Jackson	1832	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
177	Jill Stein	2016	Green	1457226	loss	1.073699
178	Joseph Biden	2020	Democratic	81268924	win	51.311515
179	Donald Trump	2020	Republican	74216154	loss	46.858542
180	Jo Jorgensen	2020	Libertarian	1865724	loss	1.177979
181	Howard Hawkins	2020	Green	405035	loss	0.255731



Column Names Are Usually Unique!

Column names in **pandas** are almost always unique.

- Example: Really shouldn't have two columns named "Candidate".

	Candidate	Party	%	Year	Result
0	Obama	Democratic	52.9	2008	win
1	McCain	Republican	45.7	2008	loss
2	Obama	Democratic	51.1	2012	win
3	Romney	Republican	47.2	2012	loss
4	Clinton	Democratic	48.2	2016	loss
5	Trump	Republican	46.1	2016	win

Learning Objective:

**Extract data from DataFrames using
.loc, .iloc and []**

Data Extraction

- Pandas Bootcamp:
 - Pandas Data Structures
 - **Extracting Data**
 - Conditional Selection
 - Adding/Modifying/Deleting Columns
 - Useful Utility Functions

Supporting Material:

More with Conditional Selection

Extracting Data

One of the most basic tasks for manipulating a **DataFrame** is to extract rows and columns of interest. As we'll see, the large **pandas** API means there are many ways to do things.

Common ways we may want to extract data:

- Grab the first or last **n** rows in the **DataFrame**.
- Grab data with a certain label.
- Grab data at a certain position.

We'll find that all three of these methods are useful to us in data manipulation tasks.

.head and .tail

The simplest scenarios: We want to extract the first or last **n** rows from the **DataFrame**.

- `df.head(n)` will return the first **n** rows of the DataFrame **df**.
- `df.tail(n)` will return the last **n** rows.

elections

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
177	2016	Jill Stein	Green	1457226	loss	1.073699
178	2020	Joseph Biden	Democratic	81268924	win	51.311515
179	2020	Donald Trump	Republican	74216154	loss	46.858542
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
181	2020	Howard Hawkins	Green	405035	loss	0.255731

elections.head(5)

	Candidate	Year	Party	Popular vote	Result	%
0	Andrew Jackson	1824	Democratic-Republican	151271	loss	57.210122
1	John Quincy Adams	1824	Democratic-Republican	113142	win	42.789878
2	Andrew Jackson	1828	Democratic	642806	win	56.203927
3	John Quincy Adams	1828	National Republican	500897	loss	43.796073
4	Andrew Jackson	1832	Democratic	702735	win	54.574789

elections.tail(5)

	Candidate	Year	Party	Popular vote	Result	%
177	Jill Stein	2016	Green	1457226	loss	1.073699
178	Joseph Biden	2020	Democratic	81268924	win	51.311515
179	Donald Trump	2020	Republican	74216154	loss	46.858542
180	Jo Jorgensen	2020	Libertarian	1865724	loss	1.177979
181	Howard Hawkins	2020	Green	405035	loss	0.255731

Label-based Extraction: .loc

Suppose we want to extract data with specific column or index labels.

```
df.loc[row_labels_list, column_labels_list]
```

Row labels

The `.loc` accessor allows us to specify the **labels** of rows and columns to extract.

Using row label list and column label list:

```
elections.loc[[1, 4, 5], ["Party", "Candidate", "Result"]]
```

Using a row label list and a column label [slice](#):

```
elections.loc[[1, 4, 5], "Year": "Party" ]
```

Using a row label and column label:

```
elections.loc[3, "Candidate"]
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
177	2016	Jill Stein	Green	1457226	loss	1.073699
178	2020	Joseph Biden	Democratic	81268924	win	51.311515
179	2020	Donald Trump	Republican	74216154	loss	46.858542
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979
181	2020	Howard Hawkins	Green	405035	loss	0.255731

Column labels

The DataFrame elections

	Party	Candidate	Result
1	Democratic-Republican	John Quincy Adams	win
4	Democratic	Andrew Jackson	win
5	National Republican	Henry Clay	loss

	Year	Candidate	Party
1	1824	John Quincy Adams	Democratic-Republican
4	1832	Andrew Jackson	Democratic
5	1832	Henry Clay	National Republican

'John Quincy Adams'

Integer-based Extraction: .iloc

Suppose we want to extract data according to its *position*.

```
df.iloc[row_integers_list, column_integers_list]
```

The `.iloc` accessor allows us to specify the **integers** of rows and columns we wish to extract.

- Python convention: The first position has integer index 0.

Using row label list and column label list:

```
elections.iloc[[1, 2, 3], [0, 1, 2]]
```

Using a row label list and a column label [slice](#):

```
elections.iloc[[1, 2, 3], 0:3]
```

Using a row label and column label:

```
elections.iloc[0, 1]
```

	0	1	2	3	4	5	
	Year	Candidate	Party	Popular vote	Result	%	
0	0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	4	1832	Andrew Jackson	Democratic	702735	win	54.574789
	...	...	...	...	...	...	

The DataFrame `elections`

Select the rows at positions 1, 2, and 3.

	Year	Candidate	Party
1	1824	John Quincy Adams	Democratic-Republican
2	1828	Andrew Jackson	Democratic
3	1828	John Quincy Adams	National Republican

Select the columns at positions 0, 1, and 2.

Year	Candidate	Party	
1	1824	John Quincy Adams	Democratic-Republican
2	1828	Andrew Jackson	Democratic
3	1828	John Quincy Adams	National Republican

Select the rows at positions 1, 2, and 3.

	Year	Candidate	Party
1	1824	John Quincy Adams	Democratic-Republican
2	1828	Andrew Jackson	Democratic
3	1828	John Quincy Adams	National Republican

Select all columns from integer 0 to integer 2.

Remember: integer-based slicing is right-end exclusive!

'Andrew Jackson'



Selection Operators in Pandas

- **loc** performs **label-based** extraction. 1st argument is rows, 2nd argument is columns:
`df.loc[row_labels, column_labels]`
- **iloc** performs **integer-based** extraction. 1st argument is rows, 2nd argument is columns:
`df.iloc[row_integers, column_integers]`
- **SHORTCUT OPERATOR FOR 3 COMMON TYPES OF SELECTIONS: []**

Only takes one argument, which may be:

- A list of **column labels**. `df[["Year", "Result"]]` (shortcut for `df.loc[:, ["Year", "Result"]]`)
- A single **column label**. `df["Candidate"]` (shortcut for `df.loc[:, "Candidate"]`)
- A **slice** of **row numbers** `df[3:7]` (shortcut for `df.iloc[3:7, :]`)

That is, `[]` is context sensitive.

Lesson Learning Objectives:

- Understand and implement methods for extracting data: `.loc`, `.iloc`, and `[ ]`.
- Understand and implement methods for conditional selection in Pandas

Conditional Selection

- Pandas Bootcamp:
 - Data Structures
 - Extracting Data
 - **Conditional Selection**
 - Adding/Modifying/Deleting Columns
 - Useful Utility Functions

Boolean Arrays

A boolean array is an array that contains only Boolean values (True or False)

```
a = np.array([True, False, True, False, True, False, False, False, False, False])
```

iClicker: What is the output of

```
a.sum()
```

Boolean Series

Boolean Series can be generated by using logical operators on Series.

Length 182 Series where every entry is either "True" or "False", where "True" occurs for every Independent candidate.

```
elections["Party"] == "Independent"

0      False
1      False
2      False
3      False
4      False
...
177    False
178    False
179    False
180    False
181    False
Name: Party, Length: 182, dtype: bool
```

True in rows 121, 130, 143, 161, 167, 174



Boolean Series/Array Input for .loc and []

- `.loc` and `[ ]` also accept boolean arrays or Series as input.
- Rows corresponding to `True` are extracted; rows corresponding to `False` are not.

	Year	Candidate	Party	Popular vote	Result	%
0	2024	Kamala Harris	Democratic	75019230	loss	48.340000
1	2024	Donald Trump	Republican	77303568	win	49.810000
2	2024	Jill Stein	Green	861155	loss	0.600000
3	2024	Robert F. Kennedy Jr.	Independent	756383	loss	0.600000
4	2024	Chase Oliver	Libertarian	650130	loss	0.400000
5	2024	Claudia De La Cruz	Party for Socialism and Liberation	165095	loss	0.100000
6	2024	Cornel West	Independent	81041	loss	0.100000
7	2020	Joseph Biden	Democratic	81268924	win	51.311515
8	2020	Donald Trump	Republican	74216154	loss	46.858542
9	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979

`elections_first_10_rows[[True, False, True, True, False, False, True, False, True, False]]`

	Year	Candidate	Party	Popular vote	Result	%
0	2024	Kamala Harris	Democratic	75019230	loss	48.340000
2	2024	Jill Stein	Green	861155	loss	0.600000
3	2024	Robert F. Kennedy Jr.	Independent	756383	loss	0.600000
6	2024	Cornel West	Independent	81041	loss	0.100000
8	2020	Donald Trump	Republican	74216154	loss	46.858542



Boolean Series/Array Input for .loc and []

Useful because boolean Series can be generated by using logical operators on Series.

Length 182 Series where every entry is "Republican", "Democratic", "Independent", etc.

```
0    False
1    False
2    False
3    False
4    False
...
177   False
178   False
179   False
180   False
181   False
Name: Party, Length: 182, dtype: bool
```

Length 182 Series where every entry is either "True" or "False", where "True" occurs for every Independent candidate.

```
elections[elections["Party"] == "Independent"]
```

	Year	Candidate	Party	Popular vote	Result	%
121	1976	Eugene McCarthy	Independent	740460	loss	0.911649
130	1980	John B. Anderson	Independent	5719850	loss	6.631143
143	1992	Ross Perot	Independent	19743821	loss	18.956298
161	2004	Ralph Nader	Independent	465151	loss	0.380663
167	2008	Ralph Nader	Independent	739034	loss	0.563842
174	2016	Evan McMullin	Independent	732273	loss	0.539546

Outputs only the rows where the Boolean Series is True



Boolean Series Input

Can also use **.loc**.

Length 182 Series where every entry is "Republican", "Democratic", "Independent", etc.

```
0    False
1    False
2    False
3    False
4    False
...
177   False
178   False
179   False
180   False
181   False
Name: Party, Length: 182, dtype: bool
```

Length 182 Series where every entry is either "True" or "False", where "True" occurs for every Independent candidate.

```
elections.loc[elections["Party"] == "Independent"]
```

	Year	Candidate	Party	Popular vote	Result	%
121	1976	Eugene McCarthy	Independent	740460	loss	0.911649
130	1980	John B. Anderson	Independent	5719850	loss	6.631143
143	1992	Ross Perot	Independent	19743821	loss	18.956298
161	2004	Ralph Nader	Independent	465151	loss	0.380663
167	2008	Ralph Nader	Independent	739034	loss	0.563842
174	2016	Evan McMullin	Independent	732273	loss	0.539546

Outputs only the rows where the Boolean Series is True

Boolean Series can be combined using various bitwise operators, allowing filtering of results by multiple criteria.

- The **&** operator allows us to apply **bool_ser_1 and bool_ser_2**

```
elections[(elections["Result"] == "win") & (elections["%"] < 47)]
```

	Year	Candidate	Party	Popular vote	Result	%
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
20	1856	James Buchanan	Democratic	1835140	win	45.306080
23	1860	Abraham Lincoln	Republican	1855993	win	39.699408
47	1892	Grover Cleveland	Democratic	5553898	win	46.121393
70	1912	Woodrow Wilson	Democratic	6296284	win	41.933422
117	1968	Richard Nixon	Republican	31783783	win	43.565246
140	1992	Bill Clinton	Democratic	44909806	win	43.118485
173	2016	Donald Trump	Republican	62984828	win	46.407862

Bitwise Operators

`&` and `|` are examples of **bitwise operators**. They allow us to apply multiple logical conditions.

If `p` and `q` are boolean arrays (or boolean **Series**):

Symbol	Usage	Meaning
<code>~</code>	<code>~p</code>	Negation of <code>p</code>
<code> </code>	<code>p q</code>	<code>p</code> OR <code>q</code>
<code>&</code>	<code>p & q</code>	<code>p</code> AND <code>q</code>
<code>^</code>	<code>p ^ q</code>	<code>p</code> XOR <code>q</code> (exclusive or)

Conditional Selection

Poll: Which of the following will select all years when Andrew Jackson was elected President?
Select all that apply:

	Year	Candidate	Party	Popular vote	Result	%
0	2024	Kamala Harris	Democratic	75019230	loss	48.340000
1	2024	Donald Trump	Republican	77303568	win	49.810000
2	2024	Jill Stein	Green	861155	loss	0.600000
3	2024	Robert F. Kennedy Jr.	Independent	756383	loss	0.600000
4	2024	Chase Oliver	Libertarian	650130	loss	0.400000
...	...	...	...	...	...	...

A

```
elections[(elections["Candidate"]=="Andrew Jackson") & (elections["Result"]=="win")]
```

B

```
elections[elections["Candidate"]=="Andrew Jackson" & elections["Result"]=="win"]
```

C

```
elections.loc[(elections["Candidate"]=="Andrew Jackson") & (elections["Result"]=="win")]
```

D

```
elections[(elections["Candidate"]=="Andrew Jackson") and (elections["Result"]=="win")]
```

E

```
elections= elections.set_index(["Candidate", "Result"])  
elections.loc[("Andrew Jackson", "win"),:]
```

Boolean series can be combined using various operators, allowing filtering of results by multiple criteria.

- **The `&` operator allows us to apply `bool_ser_1` and `bool_ser_2`**

```
elections[(elections["Result"] == "win") & (elections["%"] < 47)]
```

Note: must use parentheses for each logical operator

can also use `.loc`:

```
elections.loc[(elections["Result"] == "win") & (elections["%"] < 47)]
```

	Year	Candidate	Party	Popular vote	Result	%
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
20	1856	James Buchanan	Democratic	1835140	win	45.306080
23	1860	Abraham Lincoln	Republican	1855993	win	39.699408
47	1892	Grover Cleveland	Democratic	5553898	win	46.121393
70	1912	Woodrow Wilson	Democratic	6296284	win	41.933422
117	1968	Richard Nixon	Republican	31783783	win	43.565246
140	1992	Bill Clinton	Democratic	44909806	win	43.118485
173	2016	Donald Trump	Republican	62984828	win	46.407862

Alternatives to Boolean Series Selection

Boolean series selection is a useful tool, but can lead to overly verbose code for complex conditions.

```
elections[(elections["Party"] == "Anti-Masonic") |  
           (elections["Party"] == "American") |  
           (elections["Party"] == "Anti-Monopoly") |  
           (elections["Party"] == "American Independent")]
```

Pandas provides **many** alternatives, for example:

- `.query`
 - `.isin`
 - `.str.startswith`
 - `.groupby.filter`
- See supporting materials for examples of these other alternatives

	Year	Candidate	Party	Popular vote	Result	%
6	1832	William Wirt	Anti-Masonic	100715	loss	7.821583
22	1856	Millard Fillmore	American	873053	loss	21.554001
38	1884	Benjamin Butler	Anti-Monopoly	134294	loss	1.335838
115	1968	George Wallace	American Independent	9901118	loss	13.571218
119	1972	John G. Schmitz	American Independent	1100868	loss	1.421524
124	1976	Lester Maddox	American Independent	170274	loss	0.209640
126	1976	Thomas J. Anderson	American	158271	loss	0.194862

Query Example

Query has a rich syntax. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.query.html>

- `dataframe.query(expr)`
 - takes a query expression as a string parameter, which has to evaluate to either True or False.
- Column names with spaces or punctuations or starting with digits must be surrounded by backticks

```
elections.query("Candidate=='John Quincy Adams' | `Popular vote` > 70000000")
```

	Year	Candidate	Party	Popular vote	Result	%
0	2024	Kamala Harris	Democratic	75019230	loss	48.340000
1	2024	Donald Trump	Republican	77303568	win	49.810000
7	2020	Joseph Biden	Democratic	81268924	win	51.311515
8	2020	Donald Trump	Republican	74216154	loss	46.858542
186	1828	John Quincy Adams	National Republican	500897	loss	43.796073
188	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878



Query Example

Query has a rich syntax.

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.query.html>

- `dataframe.query(expr)`
 - takes a query expression as a string parameter, which has to evaluate to either True or False.
- Column names with spaces or punctuations or starting with digits must be surrounded by backticks

Practice: Write code using query to select all years that Andrew Jackson was elected President

Query Example

Query has a rich syntax.

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.query.html>

- `dataframe.query(expr)`
 - takes a query expression as a string parameter, which has to evaluate to either True or False.
- Column names with spaces or punctuations or starting with digits must be surrounded by backticks
- Can access Python variables with the special @ character.

```
parties = ["Republican", "Democratic"]  
elections.query('Result == "win" & Party not in @parties')
```

	Year	Candidate	Party	Popular vote	Result	%
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
11	1840	William Henry Harrison	Whig	1275583	win	53.051213
16	1848	Zachary Taylor	Whig	1360235	win	47.309296
27	1864	Abraham Lincoln	National Union	2211317	win	54.951512

Lesson Learning Objectives:

- Understand and implement methods for extracting data: `.loc`, `.iloc`, and `[]`.
- Understand and implement methods for conditional selection in Pandas
- Add, modify and delete columns in Pandas DataFrames

Transforming Columns

- Pandas Bootcamp:
 - Extracting Data
 - Conditional Selection
 - **Adding/Modifying/Deleting Columns**
 - Useful Utility Functions

Syntax for Adding or Modifying a Column

Adding a column is easy:

Use `df.assign()`

```
# Add a column named "frac_voters" that gives the fraction of voters  
who voted for this candidate.
```

```
elections = elections.assign(frac_voters = elections["%"]/100)
```

	Year	Candidate	Party	Popular vote	Result	%	frac_voters
0	2024	Kamala Harris	Democratic	75019230	loss	48.340000	0.483400
1	2024	Donald Trump	Republican	77303568	win	49.810000	0.498100
2	2024	Jill Stein	Green	861155	loss	0.600000	0.006000
3	2024	Robert F. Kennedy Jr.	Independent	756383	loss	0.600000	0.006000
4	2024	Chase Oliver	Libertarian	650130	loss	0.400000	0.004000

An Important Note: DataFrame Copies

Notice that we *re-assigned* `elections` to an updated value on the previous slide.

```
elections = elections.assign(frac_voters = elections["%"]/100)
```

By default, `pandas` methods create a **copy** of the `DataFrame`, without changing the original `DataFrame` at all. To apply our changes, we must update our `DataFrame` to this new, modified copy.

```
elections.assign(frac_voters = elections["%"]/100)
```

`elections`

	Year	Candidate	Party	Popular vote	Result	%
0	2024	Kamala Harris	Democratic	75019230	loss	48.340000
1	2024	Donald Trump	Republican	77303568	win	49.810000
2	2024	Jill Stein	Green	861155	loss	0.600000
3	2024	Robert F. Kennedy Jr.	Independent	756383	loss	0.600000
4	2024	Chase Oliver	Libertarian	650130	loss	0.400000

Our change was not applied!



Syntax for Renaming a Column

Rename a column using the (creatively named) `.rename()` method.

- `.rename()` takes in a **dictionary** that maps old column names to their new ones.

```
# Rename "frac_voters" to "fraction"  
elections = elections.rename(columns={"frac_voters": "fraction"})
```



By default, **pandas** methods create a **copy** of the **DataFrame**, without changing the original **DataFrame** at all. To apply our changes, we must update our **DataFrame** to this new, modified copy.

Syntax for Dropping a Column (or Row)

Remove columns using the (also creatively named) `.drop` method.

- The `.drop()` method assumes you're dropping a row by default. Use `axis = "columns"` to drop a column instead.

```
elections = elections.drop("fraction", axis = "columns")
```


Learning Objective

- **Manipulate and Transform Series and DataFrames using built-in methods**

Useful Utility Functions

- **Useful Utility Functions:**
 - **Python**
 - **Series**
 - **DataFrames**

Pandas **Series** and **DataFrames** support a large number of operations, including mathematical operations, so long as the data is numerical. [NumPy reference](#).

	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6
...	...	...	...	...	...	...
407423	CA	M	2022	Zayvier	5	6
407424	CA	M	2022	Zia	5	2
407425	CA	M	2022	Zora	5	3
407426	CA	M	2022	Zuriel	5	5
407427	CA	M	2022	Zylo	5	3

407428 rows x 6 columns

```
devon_count = babynames[babynames["Name"] == "Devon"]["Count"]
```

19053	5
20481	6
21016	6
24795	6
25157	13
..	..
109089	14
110093	9
112039	6
112768	9
113765	7

Name: Count, Length: 83, dtype: int64

```
np.mean(devon_count)
20.53012048192771
```

```
np.max(devon_count)
72
```



Pandas provides an enormous number of useful utility functions.

Here are a few we will use frequently in this class:

Run these functions in the notebook cells provided to understand how they work.

Useful Python Functions

```
len(series)
```

```
len(df)
```

Useful Series Utility Functions

```
series.unique()
```

```
series.sort_values()
```

```
series.value_counts()
```

```
series.isna()
```

Useful DataFrame Utility Functions

```
df.shape
```

```
df.info()
```

```
df.describe()
```

```
df.sort_values()
```

```
df.value_counts()
```

```
df.isna()
```

Match the following functions to their descriptions

1. `info`
2. `value_counts`
3. `unique`
4. `shape`
5. `describe`

- A). returns (number of rows, number of columns) of dataframe
- B). Outputs the column integer positions, column labels, data types, memory usage, and the number of non-null cells in each column of a dataframe
- C). Counts the number of occurrences of a each unique value in a series or dataframe
- D). Generates descriptive statistics of each column
- E). Returns unique values of a series

Series.value_counts()

The `Series.value_counts` method counts the number of occurrences of a each unique value in a `Series`.

- Return value is also a `Series`.

```
elections["Candidate"].value_counts()
```

```
elections["Candidate"]
```

```
0      Joseph Biden
1      Donald Trump
2      Jo Jorgensen
3      Howard Hawkins
4      Darrell Castle
...
177     William Wirt
178     Andrew Jackson
179     John Quincy Adams
180     Andrew Jackson
181     John Quincy Adams
Name: Candidate, Length: 182, dtype: object
```

```
Norman Thomas      5
Franklin Roosevelt  4
Eugene V. Debs     4
Ralph Nader        4
Andrew Jackson     3
...
Roger MacBride     1
Lester Maddox      1
Gerald Ford        1
Eugene McCarthy    1
Wendell Willkie     1
Name: Candidate, Length: 132, dtype: int64
```

dataframe.value_counts()

The `Dataframe.value_counts` method returns a Series containing the frequency of each distinct row in a `DataFrame`

- Return value is also a (multi-indexed) `Series`.

```
elections[["Year","Party"]]
```

	Year	Party
0	2020	Democratic
1	2020	Republican
2	2020	Libertarian
3	2020	Green
4	2016	Constitution
...	...	...
177	1832	Anti-Masonic
178	1828	Democratic
179	1828	National Republican
180	1824	Democratic-Republican
181	1824	Democratic-Republican

182 rows x 2 columns

```
elections[["Year","Party"]].value_counts()
```

Year	Party	
1824	Democratic-Republican	2
1836	Whig	2
1976	Republican	1
1968	Republican	1
1972	American Independent	1
...		...
1908	Republican	1
	Socialist	1
1912	Democratic	1
	Progressive	1
2020	Republican	1
Length: 180, dtype: int64		



Series.sort_values()

The `Series.sort_values` method sorts a **Series** (by default, sorted in ascending order)

```
babynames["Name"].sort_values()
380256      Aadan
362255      Aadan
365374      Aadan
394460  Aadarsh
366561      Aaden
...
232144      Zyrah
217415      Zyrah
197519      Zyrah
220674      Zyrah
400761      Zyrah
Name: Name, Length: 400762, dtype: object
```

DataFrame.sort_values()

The **DataFrame** version requires an argument specifying the column on which to sort.

```
babynames.sort_values(by = "Count", ascending=False)
```

	State	Sex	Year	Name	Count
263272	CA	M	1956	Michael	8262
264297	CA	M	1957	Michael	8250
313644	CA	M	1990	Michael	8247
278109	CA	M	1969	Michael	8244
279405	CA	M	1970	Michael	8197
...	...	...	...	...	...
159967	CA	F	2002	Arista	5
159966	CA	F	2002	Arisbeth	5
159965	CA	F	2002	Arisa	5
159964	CA	F	2002	Arionna	5
400761	CA	M	2021	Zyrus	5

400762 rows × 5 columns

By default, rows are sorted in **ascending** order.

.shape and len()

babynames

	State	Sex	Year	Name	Count
0	CO	F	1910	Mary	193
1	CO	F	1910	Helen	112
2	CO	F	1910	Dorothy	87
3	CO	F	1910	Ruth	68
4	CO	F	1910	Margaret	67
...	...	...	...	...	...
114948	CO	M	2022	Wynn	5
114949	CO	M	2022	Zephaniah	5
114950	CO	M	2022	Zephyr	5
114951	CO	M	2022	Zeus	5
114952	CO	M	2022	Zyon	5

114953 rows × 5 columns

- .shape returns the shape of a **DataFrame** or **Series** in the form (number of rows, number of columns)

babynames.shape
(114953, 5)

len(df) returns the number of rows in a dataframe

len(babynames)

Returns 114953

DataFrame.describe()

babynames

	State	Sex	Year	Name	Count
0	CO	F	1910	Mary	193
1	CO	F	1910	Helen	112
2	CO	F	1910	Dorothy	87
3	CO	F	1910	Ruth	68
4	CO	F	1910	Margaret	67
...	...	...	...	...	...
114948	CO	M	2022	Wynn	5
114949	CO	M	2022	Zephaniah	5
114950	CO	M	2022	Zephyr	5
114951	CO	M	2022	Zeus	5
114952	CO	M	2022	Zyon	5

114953 rows x 5 columns

babynames.describe()

	Year	Count
count	114953.000000	114953.000000
mean	1981.727106	33.623533
std	30.313836	65.501008
min	1910.000000	5.000000
25%	1960.000000	7.000000
50%	1989.000000	12.000000
75%	2008.000000	30.000000
max	2022.000000	1037.000000

- A different set of statistics will be reported if `.describe()` is called on a **Series**.

```
babynames["Count"].describe()
```

```
count      114953.000000
mean         33.623533
std         65.501008
min          5.000000
25%          7.000000
50%         12.000000
75%         30.000000
max        1037.000000
Name: Count, dtype: float64
```

```
babynames["Sex"].describe()
```

```
count      114953
unique         2
top          F
freq        63777
Name: Sex, dtype: object
```

.info()

Outputs the column integer positions, column labels, data types, memory usage, and the number of non-null cells in each column

babynames

	State	Sex	Year	Name	Count
0	CO	F	1910	Mary	193
1	CO	F	1910	Helen	112
2	CO	F	1910	Dorothy	87
3	CO	F	1910	Ruth	68
4	CO	F	1910	Margaret	67
...	...	...	...	...	...
114948	CO	M	2022	Wynn	5
114949	CO	M	2022	Zephaniah	5
114950	CO	M	2022	Zephyr	5
114951	CO	M	2022	Zeus	5
114952	CO	M	2022	Zyon	5

114953 rows x 5 columns

babynames.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 114953 entries, 0 to 114952
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   State   114953 non-null object
 1   Sex     114953 non-null object
 2   Year    114953 non-null int64
 3   Name    114953 non-null object
 4   Count   114953 non-null int64
dtypes: int64(2), object(3)
memory usage: 4.4+ MB
```

.unique()

The `Series.unique` method returns an array of every unique value in a `Series`.

```
babynames["Name"].unique()
```

```
array(['Mary', 'Helen', 'Dorothy', ..., 'Zyire', 'Zylo', 'Zyrus'],  
      dtype=object)
```

Supporting Materials

- Supporting Materials
 - More on Conditional Selection

Learning Objective:

- Use Boolean conditions to extract data

Conditional Selection

Conditional Selection in Pandas

Boolean Array Input for `.loc` and `[ ]`

We learned to extract data according to its **integer position** (`.iloc`) or its **label** (`.loc`)

What if we want to extract rows that satisfy a given *condition*?

- `.loc` and `[ ]` also accept boolean arrays as input.
- Rows corresponding to **True** are extracted; rows corresponding to **False** are not.

```
babynames_first_10_rows = babynames.loc[:9, :]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
5	CA	F	1910	Ruth	128
6	CA	F	1910	Evelyn	126
7	CA	F	1910	Alice	118
8	CA	F	1910	Virginia	101
9	CA	F	1910	Elizabeth	93

Boolean Array Input for .loc and []

- `.loc` and `[ ]` also accept boolean arrays as input.
- Rows corresponding to `True` are extracted; rows corresponding to `False` are not.

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
5	CA	F	1910	Ruth	128
6	CA	F	1910	Evelyn	126
7	CA	F	1910	Alice	118
8	CA	F	1910	Virginia	101
9	CA	F	1910	Elizabeth	93

`babynames_first_10_rows[[True, False, True, False, True, False, True, False, True, False]]`

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
2	CA	F	1910	Dorothy	220
4	CA	F	1910	Frances	134
6	CA	F	1910	Evelyn	126
8	CA	F	1910	Virginia	101



We can perform the same operation using `.loc`.

```
babynames_first_10_rows.loc[[True, False, True, False, True, False, True,
False, True, False], :]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
2	CA	F	1910	Dorothy	220
4	CA	F	1910	Frances	134
6	CA	F	1910	Evelyn	126
8	CA	F	1910	Virginia	101



Boolean Array Input

Useful because boolean arrays can be generated by using logical operators on **Series**.

Length 407428 **Series** where every entry is either "True" or "False", where "True" occurs for every babynames with "Sex" = "F".

```
logical operator = (babvnames["Sex"] == "F")
0                True
1                True
2                True
3                True
4                True
...
407423          False
407424          False
407425          False
407426          False
407427          False
Name: Sex, Length: 407428, dtype: bool
```

True in rows 0, 1, 2, ...

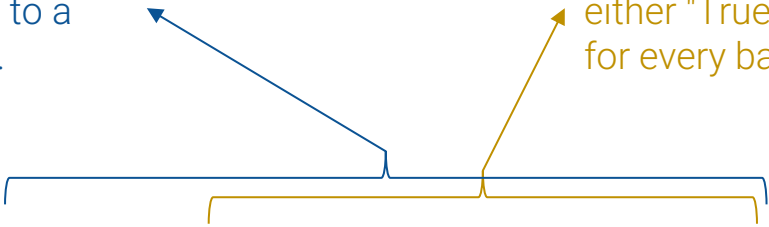


Boolean Array Input

Useful because boolean arrays can be generated by using logical operators on **Series**.

Length 239537 **DataFrame**
where every entry belongs to a
babynames with "Sex" = "F".

Length 407428 **Series** where every entry is
either "True" or "False", where "True" occurs
for every babynames with "Sex" = "F".



```
babynames[(babynames["Sex"] == "F")]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
239532	CA	F	2022	Zemira	5
239533	CA	F	2022	Ziggy	5
239534	CA	F	2022	Zimal	5
239535	CA	F	2022	Zosia	5
239536	CA	F	2022	Zulay	5

239537 rows x 5 columns

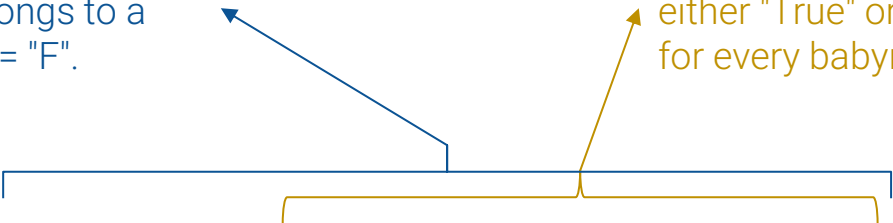


Boolean Array Input

Can also use `.loc`.

Length 239537 **DataFrame**
where every entry belongs to a
babynames with "Sex" = "F".

Length 407428 **Series** where every entry is
either "True" or "False", where "True" occurs
for every babynames with "Sex" = "F".



`babynames.loc[babynames["Sex"] == "F", :]`

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
239532	CA	F	2022	Zemira	5
239533	CA	F	2022	Ziggy	5
239534	CA	F	2022	Zimal	5
239535	CA	F	2022	Zosia	5
239536	CA	F	2022	Zulay	5

239537 rows x 5 columns

Boolean Array Input

Boolean **Series** can be combined using various operators, allowing filtering of results by multiple criteria.

- The **&** operator allows us to apply **logical_operator_1 and logical_operator_2**
- The **|** operator allows us to apply **logical_operator_1 or logical_operator_2**

```
babynames[(babynames["Sex"] == "F") & (babynames["Year"] < 2000)]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
149050	CA	F	1999	Zareen	5
149051	CA	F	1999	Zeinab	5
149052	CA	F	1999	Zhane	5
149053	CA	F	1999	Zoha	5
149054	CA	F	1999	Zoila	5

Rows that have a Sex of "F" *and* are earlier than the year 2000

49055 rows x 5 columns



Boolean Array Input

Boolean **Series** can be combined using various operators, allowing filtering of results by multiple criteria.

- The **&** operator allows us to apply `logical_operator_1` *and* `logical_operator_2`
- **The | operator allows us to apply `logical_operator_1` or `logical_operator_2`**

```
babynames[(babynames["Sex"] == "F") | (babynames["Year"] < 2000)]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134
...	...	...	...	...	...
342435	CA	M	1999	Yuuki	5
342436	CA	M	1999	Zakariya	5
342437	CA	M	1999	Zavier	5
342438	CA	M	1999	Zayn	5
342439	CA	M	1999	Zayne	5

Rows that have a Sex of "F" or are earlier than the year 2000 (or both!)

342440 rows x 5 columns



Bitwise Operators

& and | are examples of **bitwise operators**. They allow us to apply multiple logical conditions.

If **p** and **q** are boolean arrays or **Series**:

Symbol	Usage	Meaning
~	~p	Negation of p
	p q	p OR q
&	p & q	p AND q
^	p ^ q	p XOR q (exclusive or)



Alternatives to Boolean Array Selection

Boolean array selection is a useful tool, but can lead to overly verbose code for complex conditions.

```
elections[(elections["Party"] == "Anti-Masonic") |  
           (elections["Party"] == "American") |  
           (elections["Party"] == "Anti-Monopoly") |  
           (elections["Party"] == "American Independent")]
```

Pandas provides **many** alternatives, for example:

- `.isin`
- `.str.startswith`
- `.groupby.filter`

	Year	Candidate	Party	Popular vote	Result	%
6	1832	William Wirt	Anti-Masonic	100715	loss	7.821583
22	1856	Millard Fillmore	American	873053	loss	21.554001
38	1884	Benjamin Butler	Anti-Monopoly	134294	loss	1.335838
115	1968	George Wallace	American Independent	9901118	loss	13.571218
119	1972	John G. Schmitz	American Independent	1100868	loss	1.421524
124	1976	Lester Maddox	American Independent	170274	loss	0.209640
126	1976	Thomas J. Anderson	American	158271	loss	0.194862

Alternatives to Boolean Array Selection

Pandas provides **many** alternatives, for example:

- `.isin`
- `.str.startswith`
- `.groupby.filter`

```
a_parties = ["Anti-Masonic", "American", "Anti-Monopoly", "American Independent"]  
elections[elections["Party"].isin(a_parties)]
```

	Year	Candidate	Party	Popular vote	Result	%
6	1832	William Wirt	Anti-Masonic	100715	loss	7.821583
22	1856	Millard Fillmore	American	873053	loss	21.554001
38	1884	Benjamin Butler	Anti-Monopoly	134294	loss	1.335838
115	1968	George Wallace	American Independent	9901118	loss	13.571218
119	1972	John G. Schmitz	American Independent	1100868	loss	1.421524
124	1976	Lester Maddox	American Independent	170274	loss	0.209640
126	1976	Thomas J. Anderson	American	158271	loss	0.194862

Alternatives to Boolean Array Selection

Pandas provides **many** alternatives, for example:

- `.isin`
- `.str.startswith`
- `.groupby.filter`

```
elections[elections["Party"].str.startswith("A")]
```

	Year	Candidate	Party	Popular vote	Result	%
6	1832	William Wirt	Anti-Masonic	100715	loss	7.821583
22	1856	Millard Fillmore	American	873053	loss	21.554001
38	1884	Benjamin Butler	Anti-Monopoly	134294	loss	1.335838
115	1968	George Wallace	American Independent	9901118	loss	13.571218
119	1972	John G. Schmitz	American Independent	1100868	loss	1.421524
124	1976	Lester Maddox	American Independent	170274	loss	0.209640
126	1976	Thomas J. Anderson	American	158271	loss	0.194862



Filtering by Group

Another common use for groups is to filter data.

- `groupby.filter` takes an argument `func`.
- `func` is a function that:
 - Takes a `DataFrame` as input.
 - Returns either `True` or `False`.
- `filter` applies `func` to each group/sub-`DataFrame`:
 - If `func` returns `True` for a group, then all rows belonging to the group are **preserved**.
 - If `func` returns `False` for a group, then all rows belonging to that group are **filtered out**.
- Notes:
 - Filtering is done per group, not per row. Different from boolean filtering.
 - Unlike `agg()`, the column we grouped on does NOT become the index!

groupby.filter

	State	Sex	Year	Name	Count
0	CO	F	2008	Brittany	5
1	CO	F	2015	Emma	355
2	CO	F	1910	Frances	56
3	CO	F	2008	Galilea	6
4	CO	F	1910	Marie	32
5	CO	F	2015	Olivia	348

.groupby("Year")



	State	Sex	Year	Name	Count
2	CO	F	1910	Frances	56
4	CO	F	1910	Marie	32
	State	Sex	Year	Name	Count
0	CO	F	2008	Brittany	5
3	CO	F	2008	Galilea	6
	State	Sex	Year	Name	Count
1	CO	F	2015	Emma	355
5	CO	F	2015	Olivia	348

```
.filter(lambda sf: sf["Count"].sum() > 15)
```

Original DataFrame

GroupBy Object

- filter applies func to each group/sub-DataFrame:
 - If func returns **True** for a group, then all rows belonging to the group are **preserved**.
 - If func returns **False** for a group, then all rows belonging to that group are **filtered out**.

	State	Sex	Year	Name	Count
1	CO	F	2015	Emma	355
2	CO	F	1910	Frances	56
4	CO	F	1910	Marie	32
5	CO	F	2015	Olivia	348

Output DataFrame